Erasure Coding as Container Composition

what the formalism actually tells you

Robin Langer Claude

11 September 2026

Abstract

A container morphism dispatches on a shape and then routes data without ever inspecting it. Over a finite field the routing becomes a matrix, so the morphism may form *linear combinations* of its inputs — and linear combinations of data are exactly what redundancy is made of. This note is about the engineering consequence.

The claim is not that containers let you invent new codes. It is that when a system is written as a composite of container morphisms, **end-to-end recoverability becomes a rank computation on a matrix product**, decidable statically from the types, with no simulation and no per-code reasoning. Section 4 is the point of the document; everything before it is setup and everything after it is application.

Two consequences fall out that are not obvious in advance: redundancy must be introduced strictly before the loss it protects against, because rank cannot be recovered downstream; and the control plane cannot be protected by coding at all, because shapes form a set and sets have no addition.

Everything is defined. No coding-theory or networking background is assumed.

Contents

1 The engineering problem **2**

2 The formalism, in one page **2**

3 Codes, from scratch **3**

4 Composition: what the formalism tells you **4**
4.1 A pipeline is a matrix product 4
4.2 Rank cannot be recovered 4
4.3 What this buys you operationally 5

5 Real applications **5**
5.1 RAID and erasure-coded storage 5
5.2 Multicast 5
5.3 Fountain (rateless) codes 6
5.4 Random linear network coding 6

6 The control plane cannot be coded **6**

7 What is outside the formalism **7**

A Reproducing the computations **7**

1 The engineering problem

Definition 1.1 (Erasure and error)**.** A channel carries symbols and damages some of them.

- An *erasure* is a symbol that is lost, **and you know which one**.
- An *error* is a symbol that is corrupted, and you do not know which.

Almost every distributed system is an erasure channel rather than an error channel. A network packet carries a checksum; if the checksum fails the packet is discarded rather than delivered corrupted, so the receiver observes a gap, not a lie. A disk either returns a block or reports a failure. A machine is either reachable or it is not. In every case the damage announces itself.

This matters because the two failure modes sit on opposite sides of the container formalism, as §7 makes precise: erasure decoding is a container morphism and error decoding is not.

Definition 1.2 (Redundancy)**.** To survive loss you must transmit more symbols than you have message. If k symbols of message are sent as $n > k$ symbols, the $n - k$ extra symbols are *redundancy*, and the ratio n/k is the *overhead*.

There are two ways to spend that overhead, and the whole document turns on the difference.

- **Replication**: send some symbols more than once. Cheap to reason about; protects only against losing the specific symbols you duplicated.
- **Coding**: send combinations of symbols. Protects against losing *any* sufficiently small set.

In container language that difference has a name. Replication is *fan-out* — one input feeding several outputs — and it is available in **Set**. Coding is *fan-in* — one output depending on several inputs — and it is not.

2 The formalism, in one page

Definition 2.1 (Container)**.** A *container* $S \triangleleft P$ is a set S of *shapes* together with, for each shape s, a set Ps of *positions*. It denotes the datatype

$$[\![S \triangleleft P]\!]X \;=\; \sum_{s \in S} (Ps \to X) :$$

choose a shape, then label every position with a value from X.

Read S as the *control plane* — the type, the header, the metadata — and Ps as the *data plane*, the slots that carry payload.

Definition 2.2 (Container morphism)**.** A morphism $(S \triangleleft P) \to (T \triangleleft Q)$ is a pair

$$u : S \to T, \qquad f_s : Q(u\,s) \to Ps \quad \text{for each } s,$$

acting by $(s, \varphi) \mapsto (u\,s,\ \varphi \circ f_s)$.

So a morphism does two things: u reads the incoming shape and decides the outgoing shape (dispatch), and f_s says, for each outgoing position, which incoming position supplies its value (wiring). The payload is never consulted — the wiring depends on the shape alone. That restriction is what makes a morphism a finite, first-order, checkable object rather than an arbitrary program.

Definition 2.3 (q-container)**.** A *q-container* is the same with each Ps a finite-dimensional vector space over $\mathbb{F}_q$, and each f_s a *linear* map. Its denotation is $[\![S \triangleleft P]\!]V = \sum_s \mathrm{Hom}_{\mathbb{F}_q}(Ps, V)$ for V a vector space of payload values.

Fix bases. Then f_s is a matrix, and the value delivered to outgoing position j is

$$\sum_i c_{ij} \cdot (\text{value at incoming position } i),$$

the coefficients being the j-th column of that matrix.

The only difference. In **Set**, each column of the matrix would be forced to be a standard basis vector: every outgoing position copies exactly one incoming position. Over $\mathbb{F}_q$ the columns are arbitrary. A column with two or more nonzero entries is an output that *combines* two inputs — and that is redundancy. Everything in this document is a consequence of that one sentence.

Remark 2.4 (How much is gained). With two incoming and three outgoing positions over $\mathbb{F}_2$, there are 64 matrices and only 8 of them are of the **Set** kind. The other 56 are the codes.

3 Codes, from scratch

Definition 3.1 (Linear code, generator matrix)**.** A *linear code* of length n and dimension k over $\mathbb{F}_q$ — written $[n, k]$ — is a k-dimensional subspace of $\mathbb{F}_q^n$. An *encoder* is a linear injection $\mathbb{F}_q^k \to \mathbb{F}_q^n$; written in bases it is a $k \times n$ matrix G, the *generator matrix*, and the transmitted word is mG for a message row vector m.

In the container reading, G is exactly the map f_s of the encoding morphism: it has one column per transmitted symbol, and that column says which combination of message symbols the symbol carries.

Example 3.2 (Parity versus duplication)**.** Two encoders, both taking $k = 2$ message symbols to $n = 3$ transmitted symbols over $\mathbb{F}_2$, so both with the same overhead $3/2$.

$$\text{parity: } (x, y) \mapsto (x,\ y,\ x + y), \qquad \text{duplication: } (x, y) \mapsto (x,\ y,\ x).$$

Their generator columns are $\{(1,0), (0,1), (1,1)\}$ and $\{(1,0), (0,1), (1,0)\}$. Computed exhaustively:

	parity	duplication
patterns of 2 surviving symbols that recover	3/3	2/3
single losses that are fatal	none	symbol 2

Same cost, strictly different protection. Duplication defends only the symbol it duplicates; the parity symbol defends all of them. And the parity encoder is precisely the one whose third column is not a standard basis vector — the one that is not a **Set**-container morphism.

Definition 3.3 (MDS)**.** A code is *maximum distance separable* if *any* k of the n transmitted symbols determine the message. This is the best possible: you cannot recover k unknowns from fewer than k values.

Example 3.4 (Reed–Solomon)**.** Treat the k message symbols as the coefficients of a polynomial f of degree less than k over $\mathbb{F}_q$, and transmit its values $f(\alpha_1), \ldots, f(\alpha_n)$ at n distinct points. Any k of those values determine f by interpolation, so the code is MDS. The generator matrix is a Vandermonde matrix, and every transmitted symbol depends on *all* k message symbols: maximal fan-in.

For $q = 5$, $k = 3$, $n = 5$, evaluating at $x = 1, 2, 3, 4, 5$, exhaustive check:

symbols arriving	loss patterns that recover
2	0/10
3	10/10
4	5/5

Any three of the five suffice, and two never do.

4 Composition: what the formalism tells you

This is the section that earns the formalism. Everything so far could have been said without containers; what follows could not be said so cleanly.

4.1 A pipeline is a matrix product

Morphisms compose: given $(u, f) : C \to D$ and $(v, g) : D \to E$, the composite is

$$(v \circ u, \;\; s \mapsto f_s \circ g_{us}),$$

shapes forwards, positions backwards. So a system built as a chain of stages has a single *transfer matrix*, obtained by multiplying the stages' matrices, and a single *dispatch*, obtained by composing their shape maps.

Proposition 4.1 (Recoverability)**.** *Let C have position space $\mathbb{F}_q^k$ (the message) and let $h : P_E \to \mathbb{F}_q^k$ be the composite position map of a pipeline $C \to E$. A receiver holding the labelling at E can reconstruct the original message if and only if*

$$\operatorname{rank} h \;=\; k.$$

Proof. The receiver holds $\psi = \varphi \circ h$ where $\varphi : \mathbb{F}_q^k \to V$ is the message. If h is surjective, every message position is $h(e)$ for some e and $\varphi(h(e)) = \psi(e)$ determines it. If not, φ is unconstrained on any complement of $\operatorname{im} h$, and distinct messages produce identical observations. □

Corollary 4.2 (Erasures)**.** *Modelling a loss as the stage that selects the surviving coordinates, the message is recoverable exactly when* ***the surviving columns of the generator matrix span*** *$\mathbb{F}_q^k$. MDS means every k-subset of columns does.*

Note that the lossy channel itself is a **Set**-container morphism: it merely selects coordinates, and each of its columns is a standard basis vector. *Loss is pure routing. Only the protection against it requires $\mathbb{F}_q$.*

4.2 Rank cannot be recovered

Theorem 4.3 (Monotonicity)**.** $\operatorname{rank}(h_1 h_2) \le \min(\operatorname{rank} h_1, \operatorname{rank} h_2)$. *Hence along a pipeline the rank of the composite is non-increasing, and is bounded by the rank of every individual stage.*

Corollary 4.4 (The weakest stage decides)**.** *End-to-end recoverability fails as soon as any single stage has rank below k, and no subsequent stage can repair it.*

Example 4.5 (A three-stage pipeline)**.** Take the Reed–Solomon code of Example 3.4, $k = 3$, $n = 5$ over $\mathbb{F}_5$.

stage	what it does	rank of composite (need 3)
encode	$3 \to 5$, Vandermonde	3
channel	drops symbols 2 and 5	3 still recoverable
forwarder	relays only two of the three it holds	2 **lost, permanently**

The channel loss was survivable; an innocuous-looking downstream node that forwards two of three was not. Nothing downstream can help, by Theorem 4.3.

Design principle 4.6. Redundancy must be introduced strictly *before* the loss it protects against. Adding parity after a lossy stage cannot raise the rank of the composite, so it protects nothing that was already destroyed.

This is obvious once stated and routinely violated in practice — it is exactly the mistake of checksumming a value after the unreliable step that produced it.

4.3 What this buys you operationally

1. **Static analysis instead of testing.** Recoverability under a given loss pattern is a rank computation on a product of matrices, decided from the declared types of the stages. No simulated packet loss, no fault injection, no statistical confidence.
2. **Localisation of fault.** Because rank is monotone, the first stage at which the running rank drops below k is the stage responsible. That is a bisection over the pipeline, not a search over loss patterns.
3. **Heterogeneous pipelines compose.** Stages may use different codes, different field sizes after base change, and different overheads. There is one composite and one criterion, rather than a separate argument per code.
4. **A budget.** $\operatorname{rank} h - k$ at any stage is the number of further erasures that stage can still absorb. It is a resource, spent by losses and replenished only by encoding.

5 Real applications

Each is defined from scratch, then read through the formalism.

5.1 RAID and erasure-coded storage

Definition 5.1 (RAID)**.** *Redundant Array of Independent Disks*: store data across several disks with redundancy so that the array survives disk failures. RAID 5 adds one parity block computed as the XOR of the data blocks and survives one disk failure; RAID 6 adds two independent check blocks and survives two.

RAID 5 is Example 3.2 with more symbols: the $[n, n-1]$ single parity check code. RAID 6 is an $[n, n-2]$ code, usually Reed–Solomon.

Definition 5.2 (Erasure-coded storage)**.** The general version. Split an object into k blocks, encode to n blocks, place the n blocks on n different machines. Any k surviving machines reconstruct the object. A typical configuration stores 10 data blocks as 14, surviving four simultaneous machine failures at 1.4× overhead — against 3× overhead for keeping three complete copies, which survives only two failures.

The container reading: shapes are *which machines responded*, positions are the blocks. A read is the composite of the encoding morphism with the selection morphism determined by the responding set, and Proposition 4.1 is the reconstruction criterion. The comparison with replication is Example 3.2 at scale: replication is fan-out and defends only what it copied; coding is fan-in and defends against any $n - k$ failures.

5.2 Multicast

Definition 5.3 (Multicast)**.** Sending the same data from one source to many receivers at once over a shared network, as opposed to *unicast* (a separate transmission per receiver).

The difficulty is that different receivers lose different packets, so there is no single retransmission that repairs everyone. With an erasure code there need not be: each receiver decodes as soon as it holds any k symbols, and *which* k does not matter. In the formalism, one encoding morphism is composed with a different selection morphism per receiver, and the criterion is applied once per receiver against the same generator matrix.

5.3 Fountain (rateless) codes

Definition 5.4 (Fountain code)**.** The sender emits an unbounded stream of symbols, each a combination of the k source symbols; a receiver collects any k or slightly more and decodes, then leaves. *Rateless* because n is never fixed in advance.

This is the right design when the loss rate is unknown or varies per receiver: rather than guess n, you let each receiver take what it needs. In the formalism the generator matrix has unboundedly many columns, and each receiver's composite is that matrix restricted to the columns it caught. The criterion is unchanged.

5.4 Random linear network coding

Definition 5.5 (RLNC)**.** Intermediate nodes in a network do not forward packets unchanged. Each node transmits *random* $\mathbb{F}_q$-linear combinations of the packets it has received, with the coefficient vector written into the packet header. A receiver accumulates packets until it holds k independent combinations and then solves.

No central code design is required; for q large enough the random combinations are independent with high probability. This is the purest instance of the formalism: every node *is* a q-container morphism, the whole network is their composite, and the coefficient header is literally the shape — carried alongside the data precisely because the receiver must know which morphism was applied in order to invert it.

Remark 5.6. The coefficient vector travelling in the header is not an implementation detail. In the formalism the wiring depends on the shape, so the shape must accompany the data for the composite to be reconstructible at the far end. RLNC's packet format is the formalism's requirement, arrived at independently.

6 The control plane cannot be coded

Theorem 6.1. *Linear redundancy is available for positions and unavailable for shapes.*

Reason. Redundancy means forming linear combinations, which requires an addition on the values being protected. Positions span a vector space and have one. Shapes form a bare set: there is no $s_1 + s_2$. Nor can this be repaired by making the shapes linear — if the extension is taken to be $\bigoplus_s \mathrm{Hom}(Ps, V)$ rather than $\sum_s \mathrm{Hom}(Ps, V)$, the natural transformations collapse to $\mathrm{Hom}(\bigoplus_t Qt, \bigoplus_s Ps)$, which forgets the shape set entirely. □

Design principle 6.2 (Two planes, two mechanisms). Protect positions with codes. Protect shapes with replication, acknowledgement or consensus. A system that erasure-codes its payload and leaves its metadata on a single unreplicated path has not been made reliable; it has moved its single point of failure into the header.

Practice already reflects this. Packet headers are protected by checksum-and-discard, never by coding across headers; a corrupted header means the packet is dropped, not repaired. Storage systems replicate metadata while erasure-coding data. The formalism says this was not a choice.

7 What is outside the formalism

operation	a container morphism?
encoding, adding redundancy	**yes** — a matrix
transmission loss	**yes** — selection of coordinates, in **Set**
syndrome extraction $s = Hy$	**yes** — a matrix
erasure decoding, loss pattern known	**yes** — pattern in the shape, recovery in the positions
error correction, position unknown	**no** — must infer from the payload
retransmission, congestion response	**no** — reads state and branches
rate adaptation	**no** — reacts to observed loss

The line is drawn in exactly one place: a container morphism may not look at the payload. Erasure decoding survives because the loss pattern is metadata, and given the pattern the recovery is a matrix — for the $[3, 2]$ parity code, the three surviving pairs give three different recovery matrices, dispatched by u and applied by f. Error correction does not survive, because the pattern would have to be inferred from the data.

Note the split inside decoding: computing a syndrome is linear and therefore a morphism, while mapping a syndrome to an error vector is a search and therefore not. The formalism separates the algebra of decoding from the search.

Warning 7.1. None of this yields new codes. Reed–Solomon is from 1960 and network coding from 2000. What the formalism supplies is a type discipline in which a system's recoverability is a property of its declared interfaces, computable by multiplying matrices, rather than a property established per-code and per-deployment.

Summary

1. A container morphism dispatches on shape and routes data without reading it. Over $\mathbb{F}_q$ it may combine inputs linearly, which is what redundancy is.
2. A pipeline is a matrix product; recoverability is rank $= k$ (Prop. 4.1).
3. Rank is non-increasing, so the weakest stage decides and redundancy must precede loss (Thm. 4.3, Principle 4.6).
4. Recoverability is therefore decidable statically from the types, and the failing stage is localisable by bisection.
5. Positions can be coded; shapes cannot. Data and metadata require different protection mechanisms, and this is forced rather than conventional.

A Reproducing the computations

```
q-bool/code/erasure.py   examples 1-3 of this document
$ python3 erasure.py
```

All arithmetic is exact over prime fields, by exhaustive enumeration of loss patterns.